

EXPERIMENT-4 C Programming Bugging & Verification

1. Identified Flaws in Original Algorithm & Flowchart

- 1. Flawed Loop Termination Semantics:** The original flowchart specifies Repeat until (index $\leq$ num2). In formal algorithmic logic, repeating *until* a condition evaluates to True means stopping immediately when it is True. However, the flowchart incorrectly loops on True and exits on False.
- 2. Inability to Process Negative Numbers:** When the second input is negative (e.g., -5), initializing index = 1 causes the loop check index $\leq$ num2 to evaluate to False on the very first iteration. The program exits immediately and outputs 0, breaking negative integer multiplication.
- 3. Unoptimized Addition Count:** The original algorithm adds num1 repeatedly num2 times regardless of magnitude. For inputs like 12 3 versus 3 12, adding 3 twelve times takes 12 additions, failing the core requirement to

minimize additions (which requires adding 12 three times for only 3 additions).

2. Corrected Algorithm

1. Start

2. **Input:** Read integers A and B .

3. **Zero Validation:** If $A = 0$ or $B = 0$, set product = 0, print product, and go to Step 9.

4. **Determine Result Sign:** Set $\text{is_negative} = (A < 0) \text{ XOR } (B < 0)$.

5. **Absolute Value Conversion:** Set $a = |A|$ and $b = |B|$.

6. Minimize Additions:

- If $a < b$, set count = a and addend = b .
- Else, set count = b and addend = a .

7. Repeated Addition Loop:

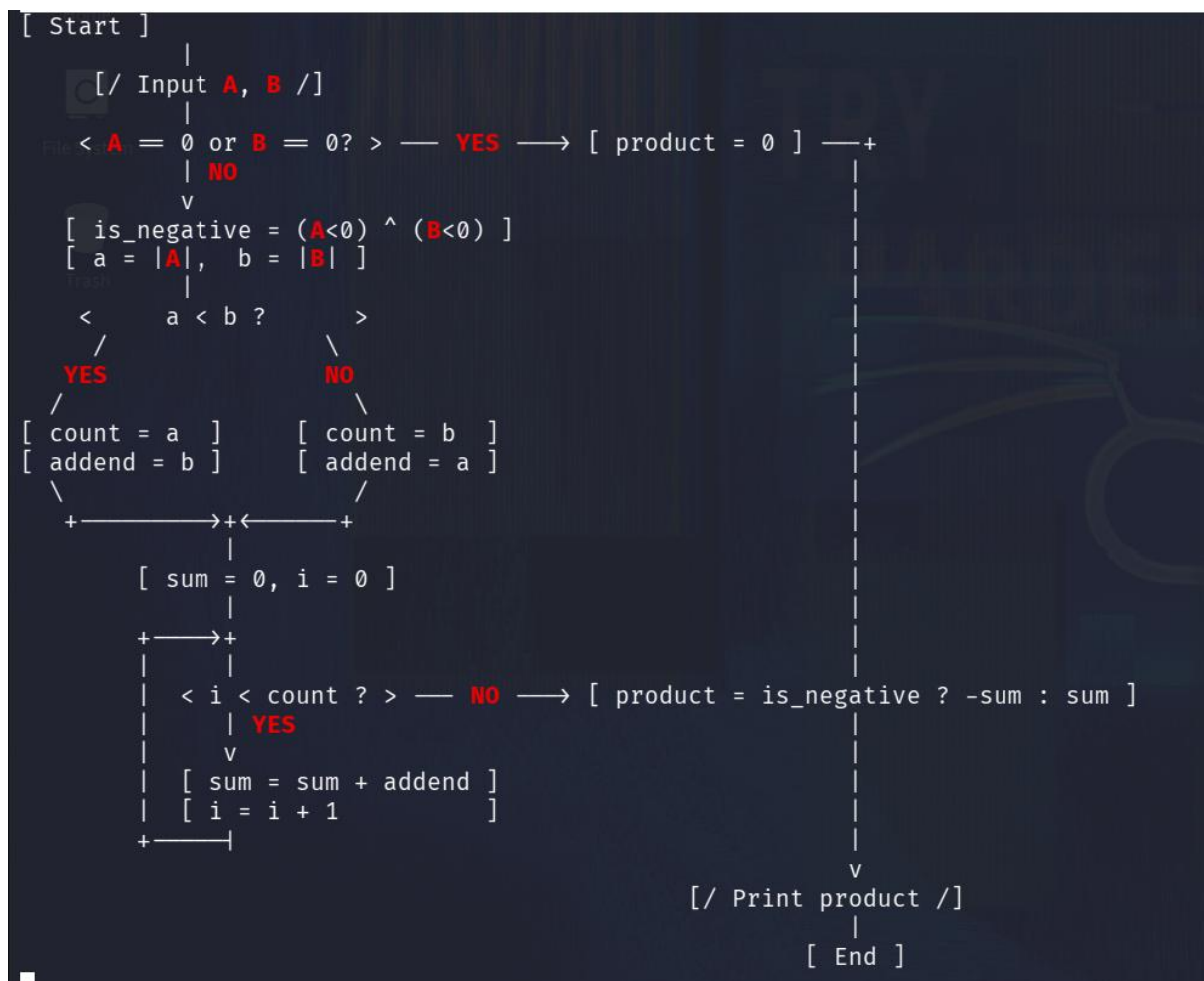
- Initialize sum = 0 and $i = 0$.
- While $i < \text{count}$:
 - sum = sum + addend
 - $i = i + 1$

8. **Apply Sign:** If `is_negative` is True, set `product = -sum`; else set `product = sum`.

9. **Output:** Print product.

10. **End**

3. Corrected Flowchart Diagram

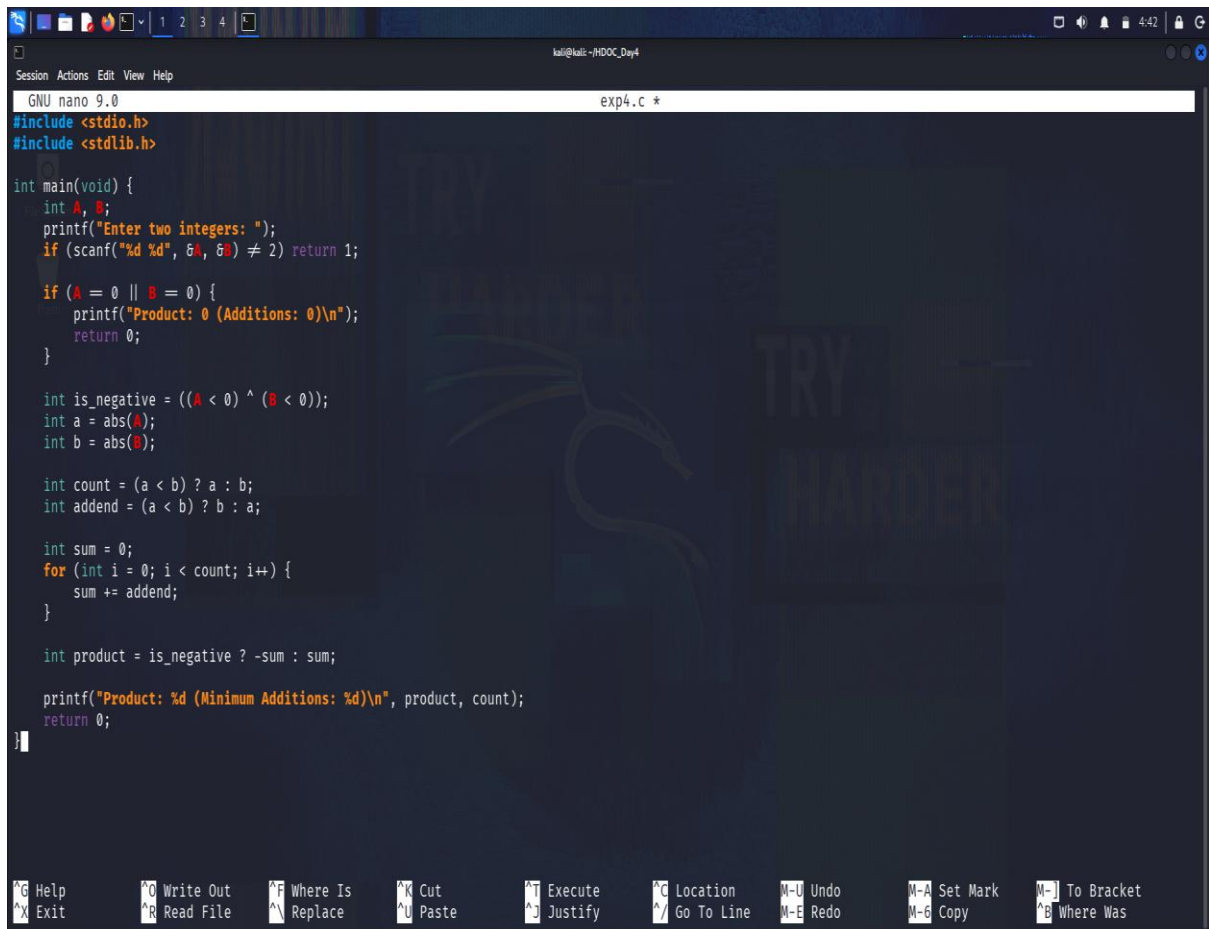


4. Verification Test Cases Table

Test Case	Input (A, B)	Expected Output	Minimum Additions
1	12 3	Product: 36	3

Test Case	Input (A, B)	Expected Output	Minimum Additions
2	3 12	Product: 36	3
3	-6 5	Product: -30	5
4	6 -5	Product: -30	5
5	-9 -4	Product: 36	4
6	-4 -9	Product: 36	4
7	0 8	Product: 0	0
8	8 0	Product: 0	0
9	0 -7	Product: 0	0
10	-7 0	Product: 0	0
11	0 0	Product: 0	0
12	1 -7	Product: -7	1
13	-1 -9	Product: 9	1

5. C Source Code Implementation

A screenshot of a terminal window with a dark blue background. At the top, there's a window title bar with icons and the text 'kali@kali: ~/HDOOC_Day4'. Below it is a menu bar with 'Session Actions Edit View Help'. The main area shows the nano 9.0 editor with a file named 'exp4.c *'. The code is a C program for calculating the product of two integers A and B using only addition and subtraction. It includes headers for stdio.h and stdlib.h. The main function prompts the user to enter two integers, checks if both are non-zero, and then calculates the product by adding or subtracting one number from the other based on their signs. A for loop is used to perform the repeated addition or subtraction. The program prints the result and the number of additions performed. At the bottom, there's a toolbar with various keyboard shortcuts like 'G Help', 'X Exit', 'O Write Out', 'R Read File', 'F Where Is', 'N Replace', 'K Cut', 'U Paste', 'T Execute', 'J Justify', 'C Location', 'G Go To Line', 'M-U Undo', 'M-E Redo', 'M-A Set Mark', 'M-C Copy', 'M-I To Bracket', and 'M-B Where Was'.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main(void) {
```

```
    int A, B;
```

```
    printf("Enter two integers: ");
```

```
    if (scanf("%d %d", &A, &B) != 2) return 1;
```

```
    // Handle zero multiplication
```

```
if (A == 0 || B == 0) {  
    printf("Product: 0 (Additions: 0)\n");  
    return 0;  
}
```

```
// Determine final sign using XOR logic  
int is_negative = ((A < 0) ^ (B < 0));
```

```
// Get absolute values  
int a = abs(A);  
int b = abs(B);
```

```
// Swap to ensure minimum additions (smaller  
number becomes loop counter)
```

```
int count = (a < b) ? a : b;  
int addend = (a < b) ? b : a;
```

```
int sum = 0;  
for (int i = 0; i < count; i++) {
```

```

        sum += addend;

    }

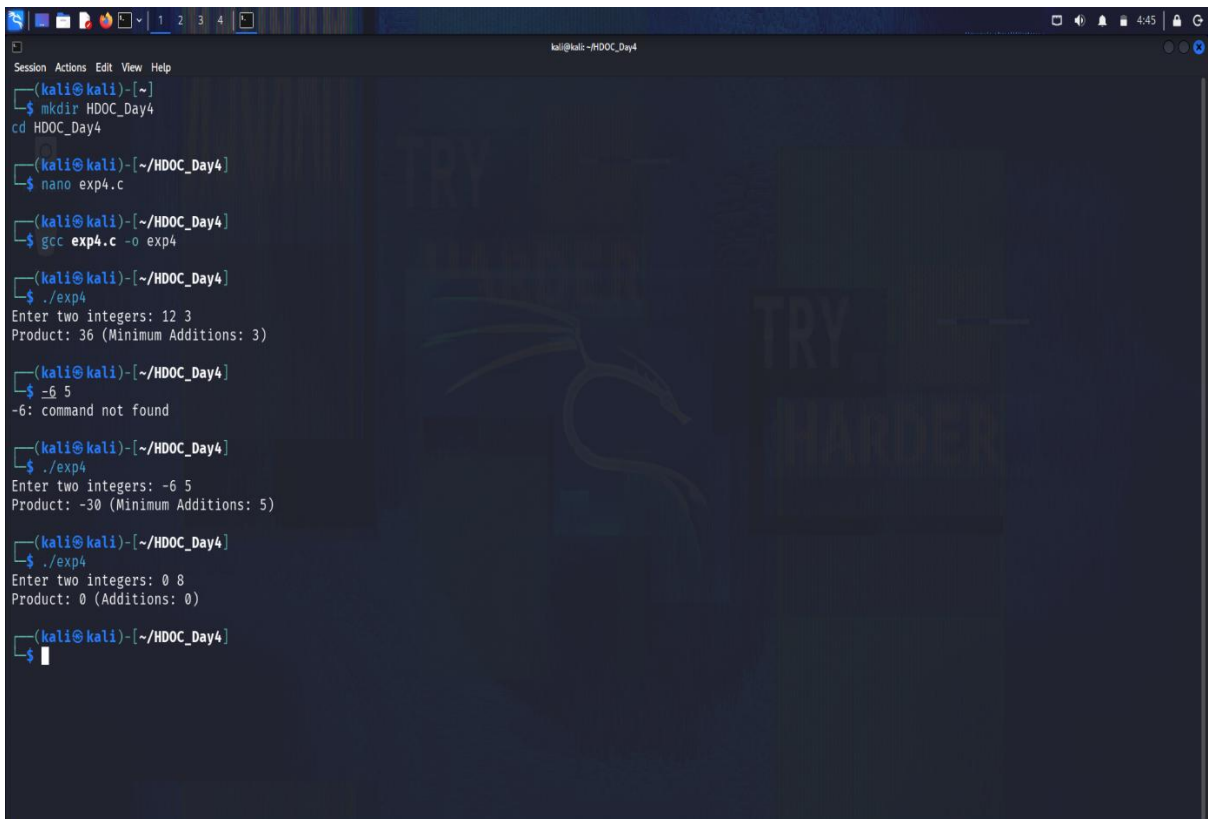
    int product = is_negative ? -sum : sum;

    printf("Product: %d (Minimum Additions: %d)\n",
product, count);

    return 0;
}

```

6. Execution & Terminal Proof



```

kali@kali: ~/HDOC_Day4
Session Actions Edit View Help
(kali@kali)-[~]
└─$ mkdir HDOC_Day4
cd HDOC_Day4

(kali@kali)-[~/HDOC_Day4]
└─$ nano exp4.c

(kali@kali)-[~/HDOC_Day4]
└─$ gcc exp4.c -o exp4

(kali@kali)-[~/HDOC_Day4]
└─$ ./exp4
Enter two integers: 12 3
Product: 36 (Minimum Additions: 3)

(kali@kali)-[~/HDOC_Day4]
└─$ ./exp4
Enter two integers: -6 5
Product: -30 (Minimum Additions: 5)

(kali@kali)-[~/HDOC_Day4]
└─$ ./exp4
Enter two integers: 0 8
Product: 0 (Additions: 0)

(kali@kali)-[~/HDOC_Day4]
└─$

```